# Práctico 01 — Intro a OpenGL y GLSL

## Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

14 de agosto de 2026



# Roadmap de la clase

**Antecedentes**

- Unidad I — sistemas gráficos
- Unidad II — algoritmos para rasterizar primitivas 2D

**Hoy**

- Cómo se dibuja y en donde hay que intervenir
- Cómo se pasan datos a la GPU
- Qué es un *shader*

**Conectado a**

- Unidad VI — transformaciones (teórico 26-ago, práctico 28-ago)
- Unidad VII — proyección (teóricos 09 y 16-sep)

## Las cajas negras de hoy

- Las coordenadas de los vértices se escriben **a mano** en el rango $-1..1$.
  - **Consecuencia**: lo que cae fuera de ese rango, no se ve
  - **Se soluciona**: el 28-ago, cuando se usen transformaciones

# La misma decisión, ahora en hardware

## En Unidad II

Trazar el segmento de $(10, 20)$ a $(300, 180)$: Bresenham decide, píxel por píxel, cuáles se prenden. Esa decisión **se escribía**: era un bucle, con su acumulador de error, en código propio.

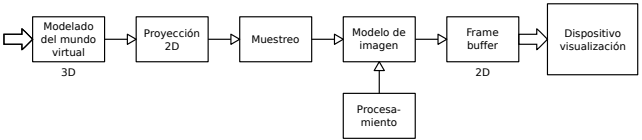
## Hoy

Se le pide un triángulo a la GPU. **La decisión es la misma** — alguien tiene que resolver qué píxeles cubre esa figura — pero ya no se escribe: está **fija en el hardware**, dentro de una etapa que no se programa.

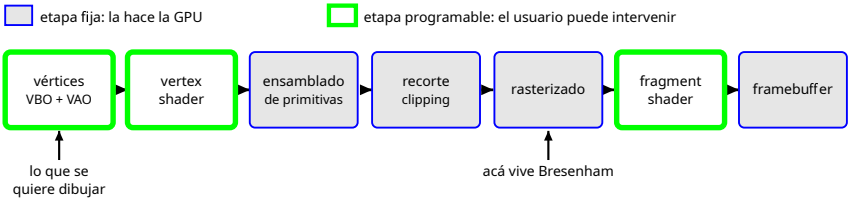
Entonces, si esa parte ya está resuelta y no se toca, ¿dónde se interviene?

# Pipeline de rendering

## El pipeline *abstracto*




## El pipeline *práctico*




Entran vértices por un lado y salen píxeles por el otro. Cada etapa recibe la salida de la anterior.

# Cómo se distribuyen las responsabilidades

## El usuario interviene en:

- Los datos: vértices y sus atributos
- El **vertex shader**: qué hacer con cada vértice
- El **fragment shader**: de qué color sale cada fragmento

## Lo hace la GPU, y no se puede cambiar

- Armar triángulos con los vértices
- Recortar lo que se sale de la pantalla
- **Rasterizar**: decidir qué píxeles cubre cada triángulo
- Escribir el resultado en el framebuffer

Tres puntos programables, cuatro etapas fijas. Todo el año va a ser **ocupar mejor esos tres**.

# ¿Qué es OpenGL?

## Una especificación, no una biblioteca

Khronos publica el **contrato**: qué funciones existen y qué tiene que hacer cada una. Quien las **implementa** es el driver de la placa de video. No hay un "OpenGL.dll" oficial que se descargue.

- Es una **máquina de estados**: las llamadas no describen un dibujo, van dejando configurado el pipeline. Recién una orden explícita lo dispara.
- **No** abre ventanas, no lee el teclado, no carga imágenes. Nada de eso está en la especificación.
- Por eso el template trae dos cosas más: **GLFW** (ventana, contexto e input) y **GLAD** (encontrar, en tiempo de ejecución, dónde implementó el driver cada función).

# Treinta años de OpenGL, en seis fechas

| | | |
|---|---|---|
| **1992** | 1.0 | El pipeline es **fijo**. Se dibuja con `glBegin` / `glVertex` / `glEnd` |
| **2004** | 2.0 | Entra **GLSL** al core: aparecen los shaders |
| **2009** | 3.1 | Se **elimina** el pipeline fijo. 3.2 parte la API en *core* y *compatibility* |
| **2014** | 4.5 | Entra **DSA** al core: cada llamada nombra el objeto sobre el que opera |
| **2016** | — | Khronos publica **Vulkan** |
| **2017** | 4.6 | Última versión de OpenGL |

En la materia se usa **4.6 core**: la última versión, y sin las partes que se eliminaron en 2009.

# Por qué no hay `drawTriangle(x1,y1,...)`

- En el OpenGL de 1992 existía algo parecido: `glBegin` / `glVertex` / `glEnd`, un vértice por llamada.
- Es lo que aparece en la mayoría de los tutoriales de libros y de internet. No compila contra un contexto 4.6 core.
- El motivo no es estético: el hardware **tiene** una cadena que procesa miles de vértices **en paralelo**, y una función que recibe un vértice por vez no puede alimentarla.
- Entonces "dibujar" se traduce a: **dejar los datos donde la GPU los vea** y después **correr el pipeline sobre esos datos**.

Las dos mitades de esa última línea son, exactamente, el bloque que viene.

# ¿Y Vulkan? ¿Y DirectX 12?

La comparación útil no es la lista de funciones. Es **quién administra qué**.

| | **OpenGL 4.6** | **Vulkan / DX12** |
|---|---|---|
| Memoria de la GPU | la administra el driver | la administra el usuario |
| Sincronización | la resuelve el driver | la resuelve el usuario |
| Estado del pipeline | global, lo modifica el usuario | compilado en objetos fijos |
| Multithreading | prácticamente no | es el punto de partida |

Las etapas del pipeline **son las mismas**. Lo que cambia es **quién administra los recursos**: los cuatro renglones de la tabla.

# El mismo triángulo

OpenGL 4.6: ∼**50 líneas** Vulkan: ∼**800 a 1000 líneas**

*líneas de llamadas a la API: sin contar la ventana, los comentarios ni el manejo de errores*

- Esas 800 líneas no son burocracia: son decisiones que en OpenGL las toma el driver.
- Más control significa más responsabilidad, y mucho más código antes del primer píxel.
- Para un motor de producción, ese control es necesario para lograr las performance esperadas. Para **aprender qué hace cada etapa**, es ruido.

Para el que tenga curiosidad, el tutorial oficial de Vulkan que llega al triángulo:
`https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/00_Setup/00_Base_code.html`

# Entonces, ¿por qué OpenGL?

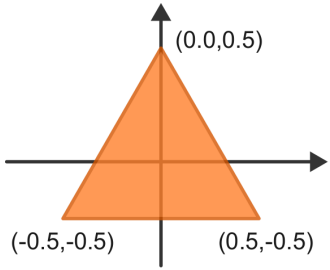
- Como se vio en el timeline, **4.6 es de 2017** y Khronos ya no evoluciona OpenGL. El sucesor es *Vulkan*.
- Sigue siendo el estándar didáctico porque es la API donde **el pipeline se ve** sin tener que agregar 800 líneas de configuración.
- Todo lo de hoy — etapas, buffers, formato de vértice, shaders — es **idéntico** en Vulkan. Lo que cambia es quién administra la memoria y la sincronización.

Se elige la API donde se ve el pipeline.

# Los datos: tres vértices

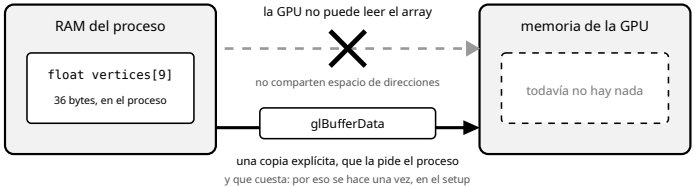
```
const float vertices[] = {
     0.0f,  0.5f, 0.0f,   // arriba
    -0.5f, -0.5f, 0.0f,   // abajo izquierda
     0.5f, -0.5f, 0.0f    // abajo derecha
};
```

- 3 vértices × 3 coordenadas = **9 floats**
- 9 floats × 4 bytes = 36 bytes




Un array común de C++, en la memoria del proceso. Todavía no interviene OpenGL.

# Ese array no existe para la GPU




Dos máquinas, dos memorias, dos espacios de direcciones. El puntero de C++ no significa nada del otro lado: hay que **pedir una copia**.

# El patrón que se repite toda la materia

`crear` → `subir`

```
GLuint vbo = 0;
glCreateBuffers(1, &vbo);                  // la GPU devuelve un nombre: un entero
glNamedBufferData(vbo, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

- Los objetos de OpenGL no son punteros: son **nombres** (`GLuint`). El objeto vive del otro lado.
- Cada llamada dice **sobre qué objeto opera**: el nombre va como primer argumento.
- Este estilo se llama **DSA** (*Direct State Access*) y está en el core desde OpenGL 4.5. Es el que se usa en la materia.

# El estilo que se va a encontrar en internet

`gen` → `bind` → `upload`

```
GLuint vbo = 0;
glGenBuffers(1, &vbo);
glBindBuffer(GL_ARRAY_BUFFER, vbo);     // queda "activo" en el contexto
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

- El objeto se deja **activo** en el contexto, y las llamadas siguientes le pegan a ése.
- `glBufferData` no menciona `vbo`: le habla al que está bindeado.
- El contexto es **uno solo y global**. De ahí sale buena parte de los errores del año: la llamada le pega al objeto que quedó activo, no al que estaba en la cabeza de quien escribe.

Las dos formas conviven en 4.6 y hacen lo mismo. Casi todo el material de internet usa ésta.

# ¿Y qué sabe la GPU de esos bytes?

Nada.

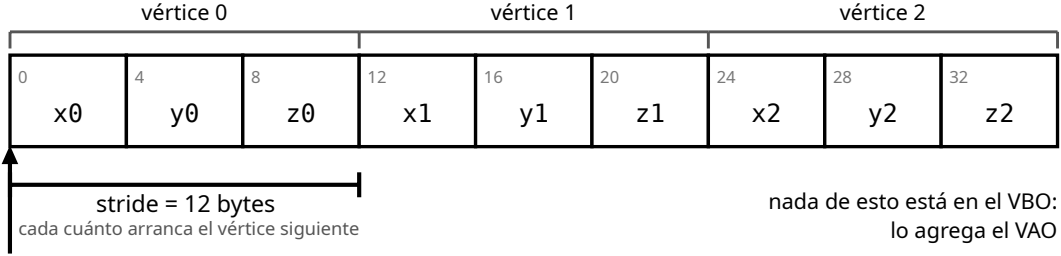
- Se copiaron los 36 **bytes** crudos del array, y nada más.
- El tipo del dato de origen, en la firma de la función, es `void`:

```
// DSA
void glNamedBufferData(GLuint buffer, GLsizeiptr size, const void *data, GLenum usage);
// clasico
void glBufferData(GLenum target, GLsizeiptr size, const void *data, GLenum usage);
```

- Los mismos 36 bytes podrían ser: 3 vértices de $(x, y, z)$, o 3 vértices de $(x, y)$ más un color, o 4 vértices de $(x, y)$ con dos números de sobra, u otra combinación cualquiera.
- **Falta decirle cómo leerlos.**

# El formato de vértice: qué hay que declarar

lo que la GPU tiene después de glNamedBufferData: 36 bytes, sin tipo




Tres preguntas: **cuántos números y de qué tipo** tiene el atributo, **cada cuántos bytes** arranca el vértice siguiente (*stride*), y **dónde arranca** el atributo dentro del vértice (*offset*).

# Dónde se guarda esa declaración: el VAO

- Esa declaración **no viaja con el buffer**: el VBO son bytes, y nada más.
- Tampoco es un dato suelto del programa: la necesita la GPU, cada vez que se dibuja.
- El **VAO** (*Vertex Array Object*) es el objeto que la guarda. Se declara una vez, en el setup, y después se reactiva con una sola llamada.

```
GLuint vao = 0;
glCreateVertexArrays(1, &vao);
```

## Dos formas de decir lo mismo

El **VBO** son los bytes. El **VAO** es el instructivo de lectura de esos bytes.

# Declarar el formato: cuatro llamadas

```
glVertexArrayVertexBuffer(vao, 0, vbo, 0, 3 * sizeof(float));
glVertexArrayAttribFormat(vao, 0, 3, GL_FLOAT, GL_FALSE, 0);
glVertexArrayAttribBinding(vao, 0, 0);
glEnableVertexArrayAttrib(vao, 0);
```

| | |
|---|---|
| `VertexBuffer` | de qué buffer se lee, y cada cuántos bytes (**stride**) |
| `AttribFormat` | cómo es el atributo: cuántos, de qué tipo, con qué **offset** |
| `AttribBinding` | qué atributo lee de qué buffer |
| `EnableAttrib` | prende la lectura desde el buffer |

Todas reciben `vao` como primer argumento: no hace falta bindear nada para configurarlo.

# Campo por campo

```
glVertexArrayVertexBuffer(vao, 0, vbo, 0, 3 * sizeof(float));
```

| | |
|---|---|
| 0 (2°) | *binding index*: la ranura donde se enchufa el buffer |
| `vbo` | qué buffer se enchufa en esa ranura |
| 0 (4°) | desde qué byte del buffer se empieza a leer |
| `3*sizeof(float)` | **stride**: cada cuánto arranca el vértice siguiente |

```
glVertexArrayAttribFormat(vao, 0, 3, GL_FLOAT, GL_FALSE, 0);
```

| | |
|---|---|
| 0 (2°) | *attrib index*: el `location` del shader |
| 3 | cuántos números tiene este atributo por vértice |
| `GL_FLOAT` | de qué tipo son |
| `GL_FALSE` | no normalizar |
| 0 (último) | **offset**: dónde arranca el atributo dentro del vértice |

Hoy son todos ceros y no son el mismo cero: un atributo, un buffer, sin offsets.

# Falta prender el atributo

```
glEnableVertexArrayAttrib(vao, 0);
```

- Cada atributo del VAO tiene un interruptor, y **arranca apagado**.
- **Apagado**: el vertex shader no lee del buffer. Recibe un valor constante, que por defecto es $(0, 0, 0, 1)$ — *el mismo para los tres vértices*.
- **Prendido**: el vertex shader lee del buffer, vértice por vértice, como se declaró recién.

## Por qué importa

Si falta esta llamada, los tres vértices caen en el mismo punto: el triángulo se degenera y no se dibuja nada. OpenGL no reporta ningún error — la pantalla queda igual que si todo estuviera bien.

# Y recién ahí, dibujar

```
glBindVertexArray(vao);
glDrawArrays(GL_TRIANGLES, 0, 3);
```

"Correr el pipeline sobre estos 3 vértices,
y armar triángulos con ellos."

DSA sacó el `bind` de la **configuración**. No lo saca del **dibujo**: `glDrawArrays` no recibe el VAO, lee del que esté activo.

Ahora ustedes: **¿en qué orden va todo esto?**

# De la ventana azul al triángulo: cuatro etapas

## 1 · Decidir los datos

la ventana no cambia

**B** — los tres vértices en un array de C++, en el rango $[-1; 1]$

## 2 · Enviar los bytes a la GPU y decir cómo se leen

la ventana no cambia

**D**/**I** — crear el VAO y el VBO **E** — `glNamedBufferData`

**G** — `glVertexArrayVertexBuffer` + `AttribFormat` + `AttribBinding` + `EnableAttrib`

## 3 · Armar el programa que los va a procesar

la ventana no cambia

**J** — escribir los dos strings **A** — compilar *y pedir el log* **F** — linkear

## 4 · Dar la orden

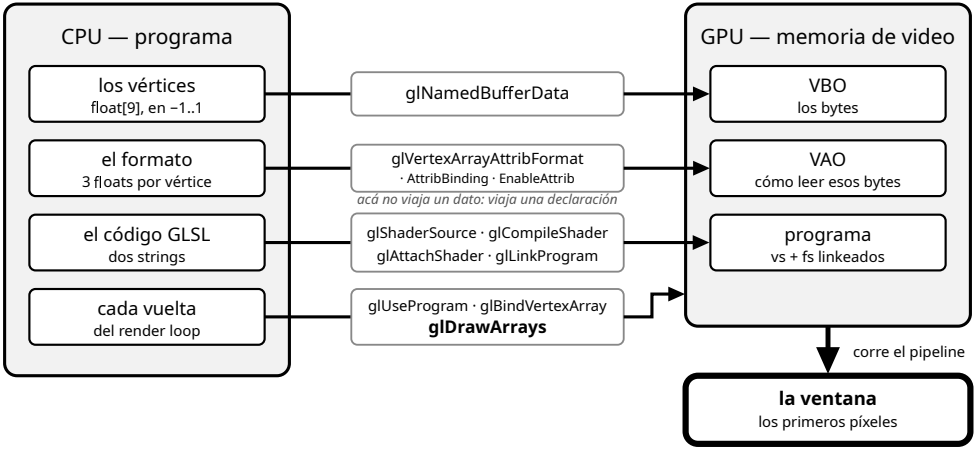
acá aparece el triángulo

**H** — `glUseProgram` **C** — `glBindVertexArray` + `glDrawArrays`

# El orden no es único — y dónde sí lo es

- Los pasos **D** e **I** se pueden dar vuelta: el VAO antes o después del VBO. Las dos respuestas están bien.
- Lo que **no** se puede alternar es **G** antes de **D** *y* de **I**.
- ¿Por qué? Porque la llamada los nombra a los dos: `glVertexArrayVertexBuffer(vao, 0, vbo, 0, 12)`. Si alguno todavía no existe, no hay nada que declarar.
  - No describe el orden de la información en el buffer
  - Describe cómo **con este VAO** se va a leer el contenido de **ese VBO**. Hacen falta los dos nombres.
- Corolario: **el formato del vértice vive en el VAO**, no en el VBO.

# Quién le habla a quién



## ¿Dónde aparece el primer píxel?

### En C — `glDrawArrays`

Los otros nueve pasos no cambian ni un píxel.

*Letra chica: en rigor los píxeles llegan a la ventana en el `glfwSwapBuffers`. El template ya lo hace en cada vuelta del loop, así que no aparece como paso de la hoja.*

# Entonces, ¿como se puede depurar?

- Durante nueve de los diez pasos, la pantalla dice **lo mismo** esté todo bien o esté todo mal.
- Que no se vea nada no es información: es lo esperado hasta el último paso.
- Mirar la pantalla no alcanza. Hay que **preguntarle a la máquina** — y hay que saber qué preguntarle.

Eso es lo que viene ahora.

# Qué es un "shader"

Un programa que corre **una vez por vértice**
o **una vez por fragmento**.

- En paralelo, muchas invocaciones a la vez.
- Cada una **no sabe nada de las demás**: no ve el vértice de al lado, ni cuántos hay, ni en qué orden salieron.
- Por eso adentro de un shader no hay un bucle sobre "todos los vértices", ni una variable que se acumule entre invocaciones.
- Se escribe el comportamiento para **una** invocación. La GPU lo ejecuta tantas veces como haga falta.

# En qué se escribe: GLSL

**GLSL** — *OpenGL Shading Language*

Parte del estándar de Khronos, en el core desde 2004 (OpenGL 2.0).

- Sintaxis **parecida a C**: tipos, funciones, `if`, `for`, un `main()`.
- Lo que **agrega**, porque es un lenguaje para hacer cuentas con geometría:
  - vectores y matrices como tipos nativos — `vec3`, `vec4`, `mat4`;
  - acceso por componente — `v.x`, `v.xyz`, `v.rgb`;
  - funciones ya incorporadas — `dot`, `normalize`, `mix`.
- Lo que **no tiene**: punteros, `new` / `malloc`, recursión, ni la librería estándar de C. Adentro de un shader no hay `printf`.

No es C, y no lo compila el compilador de C++. ¿Quién lo compila, entonces?

# Lo mismo, en otras APIs

| API | Lenguaje de shaders | |
|---|---|---|
| OpenGL | **GLSL** | el de la materia |
| Direct3D (Microsoft) | HLSL | *High Level Shading Language* |
| Metal (Apple) | MSL | basado en C++ |
| Vulkan | **SPIR-V** | formato binario; GLSL y HLSL se traducen a él |

## El nombre que confunde

Lo que acá es **fragment shader**, en Direct3D se llama **pixel shader**. El nombre de OpenGL es el más honesto: un fragmento es un *candidato* a píxel — todavía puede descartarse antes de llegar al framebuffer.

Las etapas del pipeline son las mismas (B1). Cambian el nombre y la sintaxis, no el modelo.

# La firma mínima

vertex shader

```
#version 460 core

layout (location = 0) in vec3 aPos;

void main()
{
    gl_Position = vec4(aPos, 1.0);
}
```

fragment shader

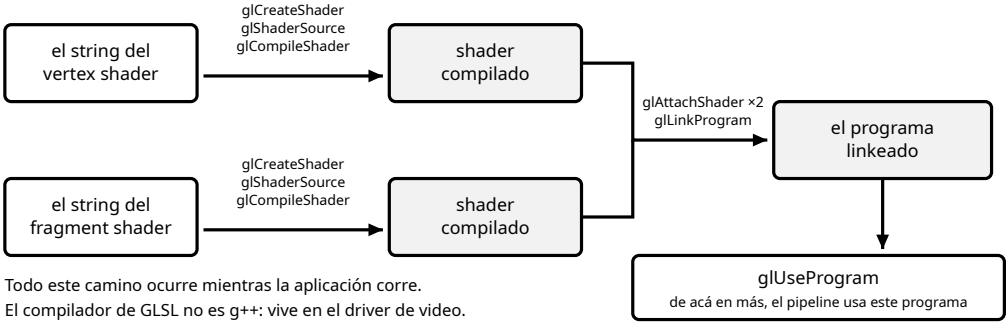
```
#version 460 core

out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0, 0.5, 0.2, 1.0);
}
```

- `#version 460 core` — primera línea, versión de la API a usar (define qué funcionalidades están disponibles).
- Listado de variables globales (algunas pueden pertenecer al pipeline de ejecución)
- Calificadores `out` e `in` permiten pasar / recibir información en el pipeline de ejecución: `out` lo que sale, `in` lo que entra
- `layout (location = 0)` — **con cuál** de los atributos declarados del lado de C++ se conecta la variable. El número tiene que coincidir.
- `main()` sin argumentos
- El vertex shader **está obligado** a escribir `gl_Position`. Si no lo hace, no hay nada que rasterizar.

# De string a programa




Todo este camino ocurre mientras la aplicación corre.
El compilador de GLSL no es g++: vive en el driver de video.

No es el único camino: desde OpenGL 4.6 se le puede entregar el shader **ya traducido** a un formato binario intermedio (**SPIR-V**), generado fuera de línea al compilar el proyecto. En la materia se usa el camino de arriba.

# "A mí me compiló bien"

- Sí: se compiló usando el compilador **de C++**. El shader es un string, y los strings siempre compilan.
- El compilador de GLSL vive en el driver de video y corre cuando la aplicación ya está andando.
- Si el shader no compila, no hay error de g++, no hay warning, no hay nada.
- **Hay que pedir el resultado de la compilación.**

```
GLint ok = 0;
glGetShaderiv(vs, GL_COMPILE_STATUS, &ok);
if (!ok) {
    GLint largo = 0;
    glGetShaderiv(vs, GL_INFO_LOG_LENGTH, &largo);  // incluye el '\0'
    std::string log(largo, '\0');
    glGetShaderInfoLog(vs, largo, nullptr, log.data());
    std::cout << "Vertex shader: " << log << std::endl;
}
```

El largo del log **se pregunta**. El `char log[512]` que aparece en casi todos los tutoriales trunca el mensaje justo cuando es largo, que es cuando más se lo necesita.

# Lo que sabemos de GLSL hasta hoy

```
#version 460 core      // siempre, versión de la API a usar

in  vec3 aPos;         // entra (por vértice)
out vec4 FragColor;    // sale

layout (location = 0) // con qué atributo
                      // se conecta

void main() { ... }   // el cuerpo de UNA
                      // invocación

gl_Position           // obligatorio en el
                      // vertex shader

vec4(aPos, 1.0)       // constructor: arma un
                      // vec4 con un vec3 + 1
```

## Otras funcionalidades

- `uniform` — 28-ago
- matrices — 28-ago
- texturas y `sampler2D` — 16-oct
- normales e iluminación — 23-oct

Cada práctico le suma renglones a esta filmina: el repertorio de GLSL crece por goteo, cuando hace falta.

Especificación completa del lenguaje (Khronos), también en el aula virtual:
`https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf`

# Predecir antes de mirar

Antes de correr el programa: **escribir qué se espera ver**.

- Si se predice "no va a cambiar nada" y no cambia nada: **se aprendió algo**.
- Si se predice "tiene que aparecer el triángulo" y no aparece: **se aprendió algo**.
- Si se corre sin predecir, la pantalla no dice nada: cualquier resultado es igual de esperable.

## La regla que vale para todo el año

Una prueba sólo informa si **podía haber fallado de otra manera**. Si el resultado es el mismo esté bien o esté mal, no se probó nada.

# Checklist de pantalla negra

1. ¿**Compiló** el shader?
   `glGetShaderiv(..., GL_COMPILE_STATUS, ...)` + `glGetShaderInfoLog`
2. ¿**Linkeó** el programa?
   `glGetProgramiv(..., GL_LINK_STATUS, ...)` + `glGetProgramInfoLog`
3. ¿Está **bindeado el VAO** en el momento de dibujar?
4. ¿Los **vértices** caen en $[-1; 1]$?
5. ¿**Winding / culling**?
   hoy `GL_CULL_FACE` está apagado: es el último sospechoso

En este orden. Las dos primeras son preguntas que se le hacen a la máquina; las otras tres, al código.

# Romperlo a propósito

- Cuando el triángulo aparezca, **no dejarlo ahí**: sacarle un punto y coma al shader y volver a correr.
- ¿Salió el mensaje del log? Entonces el instrumento funciona.
- ¿No salió nada? Entonces nunca se iba a avisar — y la próxima pantalla negra cuesta una tarde.

Un chequeo que nunca falló todavía no es un chequeo.